

HW13 — Doodlebug (Predator–Prey Simulation)

Lab Autograder Note

As this is a lab, the autograder is very general and is **not** meant to indicate full completeness. It provides only a basic evaluation that an attempt runs and produces reasonable output.

Programming Style

Pay special attention to the style of your code. Indent your code correctly, choose meaningful names for your variables, define constants where needed, choose the most appropriate control flow statements, and break down your solution into functions and classes where appropriate.

Project Goal

The goal of this project is to create a simple two-dimensional predator–prey simulation. In this simulation, the prey are ants and the predators are doodlebugs. These critters live in a world composed of a 20×20 grid of cells. Only one critter may occupy a cell at a time. The grid is enclosed, so a critter is not allowed to move off the edges of the world. Time is simulated in time steps. Each critter performs some action every time step.

Ant Rules

Ants behave according to the following model:

- **Move:** Every time step, randomly try to move up, down, left, or right. If the neighboring cell in the selected direction is occupied or would move the ant off the grid, then the ant stays in the current cell.
- **Breed:** If an ant survives for three time steps, then at the end of the time step (after moving) the ant will breed by creating a new ant in an adjacent (up, down, left, or right) empty cell. If there is no empty cell available, then no breeding occurs. Once an offspring is produced, an ant cannot produce another offspring until three more time steps have elapsed.

Doodlebug Rules

Doodlebugs behave according to the following model:

- **Move:** Every time step, if there is an adjacent ant (up, down, left, or right), then the doodlebug moves to that cell and eats the ant. Otherwise, the doodlebug moves according to the same rules as the ant. A doodlebug cannot eat other doodlebugs.
- **Breed:** If a doodlebug survives for eight time steps, then at the end of the time step it will spawn off a new doodlebug in the same manner as the ant.
- **Starve:** If a doodlebug has not eaten an ant within the last three time steps, then at the end of the third time step it will starve and die and should be removed from the grid.

Important: During one time step, all doodlebugs should move **before** the ants do.

Implementation Requirements

- Draw the world using ASCII characters: `o` for an ant, `X` for a doodlebug, and `-` for an empty space.
- Create a class named `Organism` that encapsulates basic data common to both ants and doodlebugs.
- `Organism` should have a virtual function named `move` that is defined in the derived classes `Ant` and `Doodlebug`.
- You may need additional data structures to keep track of which critters have moved during a time step.

Your Task

Initialize the world with **5 doodlebugs** and **100 ants**. After each time step, prompt the user to press `Enter` to move to the next time step. You should see a cyclical pattern between the population of predators and prey, although random perturbations may lead to the elimination of one or both species.

Submit a single .cpp. Do NOT separate this into multiple .h files. Your .cpp should contain all classes and structures needed so that it can execute when running `main`.

Example Output

An example of a program timestep output is shown below. Your output does not need to look exactly like this, but it can be a good starting point if you are confused.

```
World at time 0:
o - o o o o - - o - - - - o o - o o -
- - - - o - - o X - - - - o - - o - -
- - - - - - - o o - o - - - o - o - o
- - - - - o - - o - - - - - - - - -
- o - - - - - o - - - o - - - - - o -
- - - - - - - o o - - o o o o - - - -
- - - o o - - - o - - - - - o - - - -
- - - - - - - X - o - o - - - - o - o -
- - - - - - - o - - o - o o - - - -
- - - o - - - - - - - o o - - - - - X
- - - - o - - - - o o - o - o - - - -
- - - - - - - - o - - - - o - - - -
o - - - - - - - o - - - - - - - -
- o - o - - - o - - o - X - - - - -
o o - o - o - o - - o - - o - o - o o
- - - o - - o - - o - o - o - - o - o
o - - - - - - - o - - - o - - - - o
o - - o - - o o - o o - - - - - - o
- - o - - - X o - - - - - o - - o o o
- o - - o o o - - - - - - - - - -
Press ENTER to continue
```